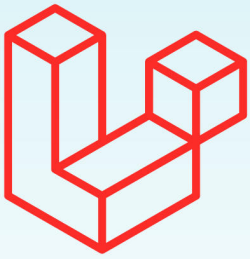
Manual desarrollo API con Laravel



Laravel
API

Miguel Angel Alvarez

desarrolloweb.com

desarrolloweb.com/manuales/desarrollo-api-laravel-sanctum-fortify

Introducción: Manual del desarrollo de API con Laravel

En este manual vamos a tratar los **primeros pasos para el desarrollo de un API o servicio web mediante Laravel**, usando por supuesto las herramientas que nos ofrece el framework PHP para la realización de este tipo de proyectos.

De entre todas las herramientas del ecosistema de Laravel las que vamos a utilizar en este manual son **Laravel Sanctum** y **Laravel Fortify**, ya que nos permiten construir un API agnóstico de los clientes que se puedan conectar. Este API por tanto no se meterá en la parte del frontend, ya que simplemente será un backend al que nos podremos conectar desde cualquier aplicación frontend que podamos necesitar.

A lo largo del manual conoceremos Laravel Sanctum y Laravel Fortify, veremos qué nos ofrecen y realizaremos los primeros pasos con nuestros proyectos Laravel para **instalar, configurar y usar los packages para desarrollar el API**.

Además veremos cómo **construir fácilmente un proyecto frontend**, un frontal web basado en Javascript, con el que podremos conectarnos con el API y realizar diversas operativas que éste nos ofrece.

Este manual está en proceso de redacción por lo que verás novedades en las próximas semanas.

Encuentras este manual online en:
https://desarrolloweb.com/manuales/desarrollo-api-laravel-sanctum-fortify

Autores del manual

Las siguientes personas han participado como autores escribiendo artículos de este manual.

Miguel Angel Alvarez

Fundador de DesarrolloWeb.com y la plataforma de formación online EscuelaIT. Comenzó en el mundo del desarrollo web en el año 1997, transformando su hobby en su trabajo.

Introducción al desarrollo de un proyecto de API con Laravel

En este artículo vamos a comenzar por establecer un poco de conocimiento básico sobre qué nos ofrece Laravel para desarrollar un API. Pondremos un poco de orden en los objetivos de este manual y todas las alternativas que te ofrece Laravel para crear ese API con autenticación que te regala el framework.

Packages del ecosistema Laravel para construir un API

Cuáles son los packages más adecuados dentro del ecosistema que nos ofrece Laravel para la construcción de un API con autenticación de usuarios.

Uno de los muchos tipos de proyectos que podemos desarrollar en Laravel sería un **API o servicio web**. Este es un modelo de proyecto muy habitual en el mundo de la web, especialmente adecuado para hacer aplicaciones modernas, donde tenemos un **backend separado del frontend**, que son dos proyectos independientes.

En el backend tenemos el API, que es capaz de recibir solicitudes HTTP a URLs, en rutas de una web a las que llamamos endpoints, para realizar operaciones y devolver una respuesta en formato JSON. Esas operaciones son generalemente las consultas, altas, bajas y modificaciones, sobre los recursos que ese API disponga.

Para consumir un API se usará un frontal, el cual hace las correspondientes conexiones a los endpoints del API por los usuarios y presentará una salida acorde con el medio donde ese frontal esté construido. Podemos tener incluso diversos frontales que usen el mismo servicio web, por ejemplo en Javascript para usarlo en una web o en Java para usarlo en una aplicación nativa de Android.

En este artículo vamos a mencionar las **herramientas que Laravel nos ofrece para la construcción de servicios web** y por qué son las más adecuadas para nuestro cometido. A lo largo de este artículo y los siguientes veremos además los pasos y configuraciones necesarios para disponer de **un proyecto Laravel para desarrollar un API backend**, a la que podremos conectar mediante cualquier tipo de frontend. Lo haremos sobre una instalación limpia del framework, construyendo desde cero un proyecto Laravel que funcionará como servicio web,

El objetivo de este proyecto sería usar el backend de Laravel desde una aplicación frontend de tipo SPA o desde una aplicación móvil.

Por qué usar Laravel Sanctum y Fortify

De entre todos los paquetes oficiales, mantenidos por el mismo equipo de Laravel, para realizar estas funcionalidades de creación de un API los más adecuados son los siguientes:

- **Laravel Sanctum**: gestiona toda la creación de tokens para API, donde el usuario es capaz de crear tokens y revocarlos. Estos token los puedes usar en cualquier tipo de aplicación, pero Sanctum además tiene la posibilidad de trabajar con sesiones, lo que puede ofrecer otras ventajas como la seguridad de CSRF.
- **Laravel Fortify**: ofrece código de backend para todas las operativas de login, registro, autenticación en dos pasos, edición del perfil y otras cosas. Sin embargo, Fortify solo se encarga de la parte del backend, siendo el desarrollador encargado de proporcionar todo el código del frontend.

Estos paquetes son a nuestro juicio los más adecuados porque son **totalmente agnósticos al frontend**. Es interesante porque el proyecto que queremos construir se basa simplemente en un API a la que cualquier tipo de frontal se podrá conectar para realizar uso del servicio web. Mediante este mismo API podríamos dar servicio a una SPA, una PWA, desarrolladas con Javascript Vanilla, Web Components, o frameworks como Angular o Vue, React, NextJS, etc. Pero también será posible usarla para dar servicio a un frontal para una aplicación móvil o de escritorio.

¿Qué tal Laravel Breeze o Jetstream?

Existen diversos starter kits en Laravel como Breeze o Jetstream. Mientras que Laravel Breeze es una extensión sencilla para aportar la parte de autenticación de usuarios en aplicaciones limpias de Laravel, Laravel Jetstream va bastante más allá, realizando no solo la parte del registro y login de usuarios, sino también un panel de administración, autenticación a dos pasos y muchas otras cosas.

Laravel Breeze o Jetstream están muy bien, solo que están dirigidos a sitios o aplicaciones web donde el mismo servidor se va a encargar tanto de la parte del frontend como del backend. Además, **estos starter kits introducen dependencias de frontend** como Tailwind, Alpine, Inertia, Livewire, etc. Nosotros no queremos nada de esto en este proyecto, ya que los frontales que se irán realizando serán proyectos totalmente independientes de este backend.

Instalación de un proyecto nuevo con Laravel

Para la construcción de un API usando Laravel Sanctum y Laravel Fortify es muy recomendable **comenzar en un proyecto nuevo de Laravel**, ya que las dependencias que vamos a instalar no queremos que causen conflictos con personalizaciones que se hayan realizado al framework a lo largo del tiempo.

> Si ya tenemos un proyecto de Laravel que estamos usando como servicio web o API realmente no nos aportará mucho Sanctum o Fortify, puesto que seguramente ya tengamos todo lo que nos hace falta para cubrir sus utilidades, como la autenticación de usuarios.

Podemos crear un nuevo proyecto de Laravel de cualquier manera que deseemos, desde una instalación clásica de Laravel hasta usando Sail si preferimos dockerizar nuestro entorno de desarrollo.

> En este punto no vamos a explicar cómo crear tus proyectos Laravel, dado que se ha visto anteriormente en DesarrolloWeb.com:
>
> - Crear un proyecto Laravel con Composer
> - Crear un proyecto Laravel mediante el instalador laravel/installer

En la actualidad yo recomendaría **crear el proyecto Laravel usando Sail**, que nos permite dockerizar el entorno de desarrollo y gracias a ello conseguimos mucha más agilidad a la hora de construir todo lo que nos hace falta para desarrollar, tanto el servidor web, como la base de datos, gestores de colas, capturadores de emails, etc. Puedes aprender en la Guía de Laravel Sail.

De todos modos, al final da un poco igual cómo desarrolles con Laravel, puedes usar el servidor integrado perfectamente, igual que una máquina Homestead. Lo importante es que hayas construido tu nueva aplicación Laravel desde cero antes de ponernos a instalar Laravel Sanctum o Laravel Fortify.

En el siguiente artículo conoceremos más de cerca sobre Laravel Sanctum y veremos cómo lo vamos a instalar.

> Este artículo es obra de *Miguel Angel Alvarez*
> Fue publicado / actualizado en *07/02/2023*
> Disponible online en *https://desarrolloweb.com/articulos/packages-laravel-construir-api*

Sanctum

En estos artículos vamos a presentar la primera de las piezas que necesitarías para construir el API con autenticación con Laravel. Se trata de Sanctum. Vamos a ver qué es Sanctum, qué nos ofrece y cómo lo vamos a configurar en un nuevo proyecto de Laravel.

Qué es Laravel Sanctum

En qué consiste Laravel Sanctum. Cuáles son sus alternativas de trabajo, basadas en sesiones o en tokens de APIs. Cómo instalar Sanctum.

En este artículo dedicado del Manual de desarrollo de API con Laravel, vamos a ofrecer una introducción a Laravel Sanctum y veremos cuáles son los primeros pasos para la instalación en un proyecto Laravel.

Qué es Laravel Sanctum

Comenzamos explicando qué es Laravel Sanctum, para los desarrolladores que no sepan encajar exactamente este producto dentro del ecosistema Laravel.

Laravel Sanctum es una capa de autenticación para una aplicación basada en API desarrollada bajo el framework Laravel. Sirve para **proyectos web SPA así como proyectos de aplicaciones móviles**, en los que se realiza la autorización por medio de tokens. Mediante Sanctum es posible generar múltiples token por usuario para autenticarse en los distintos frontales y los éstos pueden relacionarse con distintos niveles de permisos o ámbitos de acciones, llamados frecuentemente scopes.

Con Sanctum tenemos dos tipos de modelo de trabajo:

Autenticación para una SPA

Para un proyecto de SPA (Single Page Application) es posible usar Sanctum de una manera sencilla para la gestión de la autenticación por medio de **sesiones basadas en Cookies**, como en las aplicaciones web en general. Esto permite eliminar las complejidades y particularidades del trabajo con tokens de API, permitiendo que el trabajo del desarrollo de un backend se realice de manera idéntica a como lo realizarías en una aplicación Laravel tradicional.

Este modelo de trabajo solamente se puede usar desde el medio web, cuando el cliente es el navegador, ya que es quien tiene la posibilidad de trabajar con sesiones basadas en cookies. No obstante, aunque se trabaje por medio de sesiones y cookies, esto no implica que los proyectos deban residir en el mismo repositorio. Puedes tener perfectamente dos proyectos separados, uno del backend con Laravel para la implementación del API y otro proyecto frontend totalmente independiente, en el que podríamos usar cualquier tecnología o framework frontend, del estilo de Nuxt, Next, Angular, Web Components, o cualquier otra alternativa de desarrollo basada en Javascript.

Igualmente, **aunque estés desarrollando una SPA, nada te obliga a autenticarte por medio de sesiones**, ya que Laravel Sanctum también implementa el modo de trabajo basado en tokens.

Autenticación basada en tokens

Con Laravel Sanctum puedes trabajar también usando **tokens de API**, de una manera similar a como trabajan en las aplicaciones de GitHub, donde existe un **Personal Access Token** que el usuario puede generar y le permite autenticarse en su cuenta desde otras aplicaciones.

Los token se gestionan por medio de Sanctum y permiten trabajar **sin la necesidad de usar sesiones**, que es la manera más habitual de trabajo en las API, pero sin llegar a la complejidad de la capa de autenticación basada en OAuth.

Los token generados desde Sanctum pueden configurarse con un tiempo de validez, que se puede establecer si se desea de manera amplia (por ejemplo años). El usuario puede por supuesto revocar sus token en el momento que considere necesario.

Este modelo de autorización es el que **se debería usar en aplicaciones móviles**, aunque nada impide usarlo también en aplicaciones web tipo SPA.

Tal como está desarrollado Laravel Sanctum el trabajo con tokens es muy sencillo, incluso más sencillo que el trabajo mediante sesiones. Solo que mediante sesiones tienes la posibilidad de usar Laravel Fortify y con los tokens tendrías que realizar tú mismo los endpoints para el registro de usuarios y la autenticación.

En las aplicaciones frontend generalmente al hacer login o registrar usuarios recibiremos los tokens generados en el backend y los usaremos en cada una de las solicitudes al servidor para poder verificar la identidad del usuario que está realizando la solicitud.

¿Qué escoger, autenticación basada en sesiones o autenticación basada en tokens?

Para decidir uno de los dos modelos de autorización tenemos que analizar **qué tipo de aplicación se está desarrollando** y la cantidad de tráfico que vamos a recibir, lo que impacta en sus necesidades de escalabilidad.

Como primera evaluación deberíamos saber que si necesitamos implementar un frontal para una **aplicación móvil sería necesario trabajar con tokens de API**, ya que no sería posible autenticación por medio de sesiones.

Si estamos desarrollando un **frontal para web, en una ASP o PWA, podríamos escoger** entre la alternativa basada en tokens o la basada en sesiones. Siempre sabiendo que trabajar con sesiones tiene la ventaja de ser más natural y evitar toda la gestión de creación de los token.

En cuanto a la escalabilidad, si bien es cierto que el modelo basado en token resulta más sencillo de escalar en un principio y es más versátil, no es un inconveniente usar APIs basados en sesiones en la mayoría de las aplicaciones web. Obviamente, si estuvieras desarrollando un servicio como Twitter o GitHub igual sería mejor optar por los token de una vez, pero es una situación extrema que no ocurre en la generalidad de los proyectos que se desarrollan al día a día. Paralelamente, **no existe problema en gestionar en el API autorizaciones basadas en sesiones y basadas en token al mismo tiempo** y esto lo hace Sanctum por nosotros de manera predeterminada.

Otra cosa a tener en cuenta y a favor de la autenticación basada en sesiones es que es posible beneficiarse de algunas capas de seguridad implementadas en el framework Laravel como CSRF y una protección extra contra ataques por XSS.

Instalación de Laravel Sanctum

En las versiones más recientes de Laravel, **Sanctum ya se encuentra instalado de manera predeterminada**, por lo que no necesitamos hacer nada en especial en este punto. Puedes comprobarlo en el archivo composer.json de tu instalación de Laravel.

> De todos modos podemos consultar la documentación de Sanctum **para la versión de Laravel que estés utilizando**, para encontrar los pasos para la instalación si fuera necesario realizar alguno.

En el siguiente artículo aprenderemos cómo configurar Laravel Sanctum y dar los primeros pasos para el desarrollo de la capa de autenticación basada en sesiones para un proyecto de API.

> Este artículo es obra de *Miguel Angel Alvarez*
> Fue publicado / actualizado en *23/08/2023*
> Disponible online en *https://desarrolloweb.com/articulos/que-es-laravel-sanctum*

Configurar Laravel Sanctum

Qué cambios necesitamos hacer sobre la instalación inicial de Laravel para configurar Laravel Sanctum para la autenticación basada en sesiones y cookies (stateful).

En el artículo anterior ya explicamos qué es Laravel Sanctum y vimos que las versiones modernas de Laravel ya lo incorporan de serie, por lo que podemos **empezar a usar esta capa de autenticación para API** nada más iniciar un proyecto Laravel.

En este artículo vamos a ver **cómo configurar Sanctum**. Dentro de las opciones de trabajo en este artículo vamos a cubrir la "**Autenticación para SPA**", que está basada en sesiones.

> Como hemos dicho, Sanctum también podría basarse en tokens de API, lo que sería ideal para aplicaciones para móviles o escritorio, o para aplicaciones web SPA donde queremos facilitar la escalabilidad del backend. Pero si no tenemos todavía un problema de escalabilidad, o no necesitamos todavía ofrecer el backend para un móvil, sería adecuado usar el modelo de autenticación "stateful" de Sanctum, ya que nos permite algunas capas de seguridad extra. Además, el mismo paquete Sanctum nos permitiría añadir la gestión de los token de API, por lo que podemos estar tranquilos, puesto que no tendremos que deshacer nuestro trabajo en el futuro.

Solo ten en cuenta que para usar la autenticación basada en sesiones **el sitio web frontend debe compartir el dominio con el sitio web donde tienes desplegado el servicio web o API**. Es decir, el frontend tiene que compartir dominio con el backend, o bien ser un subdominio del backend. Por ejemplo, podrías tener el backend en desarrolloweb.com y el frontend en app.desarrolloweb.com. O tener desplegados ambos proyectos en el mismo dominio desarrolloweb.com. Siendo que en modo de desarrollo es perfectamente posible trabajar con localhost en tu frontend, en cualquier puerto.

> Para que quede claro, una configuración que no sería posible sería tener el backend en backend.desarrolloweb.com.es y el frontend en frontend.desarrolloweb.com, porque el frontend

no es dependiente del subdominio del backend.

Configurar el dominio o dominios donde vamos a autenticar mediante sesiones

Comenzaremos configurando nuestra aplicación para informar **desde qué dominios o subdominios vamos a poder realizar autenticación basada en sesiones**. Para comenzar abriremos el archivo `config/sanctum.php` y veremos que se hace uso de una variable de entorno llamada `SANCTUM_STATEFUL_DOMAINS`. En ella vamos a indicar dónde está el servidor frontend desde el que vamos a usar el API Backend.

Suponiendo que nuestro frontend está en http://localhost:8000, entonces editamos el archivo `.env` y colocamos la siguiente línea.

```
SANCTUM_STATEFUL_DOMAINS=localhost:8000
```

Activar el middleware de Sanctum

En el archivo de los middlewares: `app/Http/Kernel.php` tenemos que activar el middleware de Sanctum para las autenticaciones basadas en sesiones (stateful).

Para ello encontraremos una declaración como esta:

```
'api' => [
    // \Laravel\Sanctum\Http\Middleware\EnsureFrontendRequestsAreStateful::class,
    \Illuminate\Routing\Middleware\ThrottleRequests::class.':api',
    \Illuminate\Routing\Middleware\SubstituteBindings::class,
],
```

Esa declaración en versiones anteriores a Laravel 10 puede tener un código ligeramente diferente.

Pero al final la operación que vas a tener que realizar es la misma, descomentar una línea.

Simplemente tenemos que descomentar la línea del Sanctum Middleware, por lo que pasaremos a dejar este código:

```
'api' => [
    \Laravel\Sanctum\Http\Middleware\EnsureFrontendRequestsAreStateful::class,
    \Illuminate\Routing\Middleware\ThrottleRequests::class.':api',
    \Illuminate\Routing\Middleware\SubstituteBindings::class,
],
```

Activar Cors

Ahora viene la configuración que más dolores de cabeza nos puede dar. Consiste en **activar "cors" para nuestro servicio web**, de modo que se pueda acceder a él desde otros dominios. Esto es importante porque nuestro backend estará en un servidor distinto y en principio las solicitudes http

no funcionarían si no están activadas las correspondientes cabeceras del HTTP para permitir el Intercambio de Recursos de Origen Cruzado (CORS).

> La gestión de Cors la tenemos que activar desde el backend, pero también es importante cumplir una serie de normas en el frontend para poder consumir correctamente el servicio web. De momento en este artículo vamos a centrarnos en configurar el proyecto Laravel y luego en un artículo más adelante veremos cómo realizar las configuraciones en el frontend.

Para permitir Cors tenemos que llevar en consideración:

- Que el dominio desde donde queremos acceder a nuestra aplicación esté dentro de los permitidos.
- Que las rutas de la aplicación tengan activado cors.

> Recuerda que no puedes tener las aplicaciones en subdominios de dominios distintos, a no ser que el front sea un subdominio dependiente del dominio o subdominio del backend.

La parte de activar el dominio ya la hemos hecho en un paso anterior gracias a la variable de entorno `SANCTUM_STATEFUL_DOMAINS`. Ahora tenemos que configurar en el backend también el archivo `config/cors.php`.

En el archivo de configuración cors vamos a encontrar una línea como esta;

```
'paths' => ['api/*', 'sanctum/csrf-cookie'],
```

Esa línea tiene que cubrir todas las rutas del proyecto donde vamos a activar cors. Una opción podría ser dejarlo con un simple asterisco:

```
'paths' => ['*'],
```

Pero también podemos especificar todas las rutas, como por ejemplo:

```
'paths' => ['api/*', 'sanctum/csrf-cookie', 'login', 'register', 'logout'],
```

> Dependiendo de los servicios que actives del sistema de autenticación podrías necesitar otras rutas adicionales en ese array.

También en el archivo de configuración de Cors, config/cors.php, tenemos que realizar otro cambio. Se basa en la configuración supports_credentials, que tenemos que poner a true.

```
'supports_credentials' => true,
```

Permitir cookies de sesión en el dominio adecuado

En el archivo de configuración `config/session.php` tenemos otro valor que probablemente necesitemos cambiar. Se trata del "*Session Cookie Domain*", al que tendremos que asignar el valor del **dominio o subdominio donde se permitirá hacer uso de cookies de sesión**.

Verás que se usa una variable de configuración llamada `SESSION_DOMAIN`. En ella puedes colocar el dominio o subdominio desde donde quieres hacer uso de cookies de sesión.

Para ello podemos poner en nuestro `.env` el siguiente valor de configuración.

```
SESSION_DOMAIN=localhost
```

> `SESSION_DOMAIN` debe de ser .localhost o algo como .desarrolloweb.com para que acepte todos los subdominios dependientes de ese nombre de dominio.

Con estos pasos hemos conseguido configurar el package Laravel Sanctum para el uso basado en cookies de sesión. A partir de aquí podemos seguir con nuestro siguiente paso, que será instalar el paquete Laravel Fortify que nos ofrece el código necesario para implementar las rutas del API donde se realizará el proceso de autenticación. Eso lo veremos en el próximo artículo del manual.

Este artículo es obra de *Miguel Angel Alvarez*
Fue publicado / actualizado en *16/02/2023*
Disponible online en *https://desarrolloweb.com/articulos/configurar-laravel-sanctum*

Fortify

Ahora vamos a conocer el segundo paquete que usaremos para desarrollar la parte del backend del API con autenticación. Se trata de Laravel Fortify, que es el proyecto que permite tener toda la lógica de la operativa de autenticación avanzada.

Laravel Fortify

Qué es Laravel Fortify, el paquete oficial de Laravel para construir las rutas y controladores para la autenticación de usuarios, especialmente adecuado si estás desarrollando un API con Laravel.

En este artículo vamos a presentar un **package de Laravel llamado Fortify**, que nos permite componer todos los comportamientos del sistema de **login de usuarios de Laravel**, pero sin introducir ningún código de la parte de del frontend. Además podremos aprender cuáles son los primeros pasos una vez que ya tenemos claro que necesitamos usar Fortify en un proyecto Laravel.

Qué es Laravel Fortify

Laravel Fortify nos permite **implementar la lógica de autenticación de aplicaciones Laravel de una manera absolutamente independiente del frontend**. Es un paquete que instalaremos opcionalmente, si es que necesitamos autenticación en un proyecto y se encargará de registrar las rutas, controladores y otras piezas necesarias para autenticar usuarios en una aplicación.

Además, ofrece toda una serie de rutas extra, como las que serían necesarias para recordar la clave de los usuarios, la parte de verificación del email del usuario y otras cuestiones relacionadas. Incluso, aparte de todo lo anterior, también permite habilitar el doble factor de autenticación (Two Factor Authentication), cada vez más popular en las aplicaciones web, de una manera cómoda.

Fortify no introduce código del frontend

Fortify tiene la característica de no incluir ningún código frontend, al contrario que otros packages de Laravel, como Breeze o Jetstream. Fortify simplemente nos dará el código necesario para que el backend reciba las solicitudes de login o registro, y demás operativas de autenticación de usuarios, siendo nosotros los responsables de crear en nuestra aplicación frontend toda la operativa necesaria para comunicar con el usuario, como formularios, realizar las solicitudes, recibir las respuestas, mostrar el feedback al usuario, etc.

> Por supuesto que si tenemos que hacer el código frontend tendremos un trabajo extra. Para ello tenemos los starter kits de Laravel, de los que hablaremos enseguida.

Por qué usar Laravel Fortify

El uso de Laravel Fortify es perfectamente opcional en una aplicación Laravel. De hecho hay paquetes como Laravel Breeze o Jetstream que serían más frecuentes en las aplicaciones Laravel habituales y que nos ahorran mucho más tiempo de desarrollo, ya que incluyen diversos "*starter kits*" con más piezas necesarias para el trabajo en la parte de autenticación, como por ejemplo los formularios para hacer el registro, el login, hacer el reset de la clave, etc.

Entonces ¿Por qué querríamos usar Fortify? Podrían haber distintos motivos, aunque principalmente:

- En un sitio web podrías preferir que Laravel no te proporcione nada de código de la parte de frontend, ya que Laravel introduce diversas **dependencias de frontend que son perfectamente sustituibles por otras de tu preferencia**. Por ejemplo, Laravel usa Tailwind CSS, pero quizás tú prefieras hacer el CSS a mano. Por ejemplo, Laravel te instala cosas como Alpine para el Javascript que quizás tú no deseas en tu proyecto. Lo cierto es que Laravel tiene diversos *starter kits*, tanto en Breeze como en Jetstream que usan diversos stack de tecnologías frontend (con React, Vue, Livewire, Inertia), por lo que tienes bastantes opciones para elegir, sin embargo nadie te obliga a irte por ninguno de esos caminos.
- Si estás **desarrollando un API** es muy probable que no desees tener ningún código frontend en el proyecto que va a implementar el servicio web. Ese proyecto tendrá únicamente los endpoints del API pero nada de formularios de autenticación o cosas similares.

> Para sitios web es muy frecuente acabar usando un starter kit, aunque sea algo sencillo como Laravel Breeze y acabes reformulando parte del código y tus dependencias Frontend por otras más acordes a tus gustos, ya que los starter kits te ofrecen un buen punto para empezar bastante más sencillo y rápido. Donde verdaderamente le sacas más partido a Fortify es justamente en el desarrollo de proyectos de API. Luego tendrás una SPA / PWA / Movil App que estará

generalmente en un proyecto aparte, donde eligirás las tecnologías y desarrollarás la personalización completa de la experiencia de usuario.

Fortify y Sanctum

Los packages **Laravel Fortify y Laravel Sanctum irán muchas veces de la mano**, ya que se complementan perfectamente.

Sanctum realiza toda la parte de la capa de autenticación de usuarios, ya sea por cookies y sesiones o por tokens de API Fortify ofrece toda la lógica y las rutas de autenticación para que la autenticación se realice

Debemos saber que Sanctum no registra ninguna ruta de la aplicación para permitir acciones necesarias como el login o el registro de usuarios. Para crear todas las rutas y comportamientos necesarios podríamos manualmente crear toda la serie de rutas y controladores que nos harían falta, pero también podemos apoyarnos en Laravel Fortify, lo que resultará mucho más conveniente y ágil.

Fortify es por debajo lo que usa Laravel Jetstream para funcionar. Solamente que Jetstream también te aporta toda la parte del código frontend.

Instalación de Laravel Fortify

Fortify no viene instalado de casa en los proyectos iniciales de Laravel. Para instalar Fortify tendremos que realizar una serie de acciones que vamos a resumir.

Comenzamos realizando la **instalación de la dependencia mediante Composer**. Para ello usamos el siguiente comando.

```
composer require laravel/fortify
```

Recuerda que si estás usando Laravel Sail todos los comandos siguientes serían con "`sail`" delante, algo como sail composer require laravel/fortify

Luego tenemos que publicar los archivos que permiten controlar las rutas de Fortify, para el login, registro, etc.

```
php artisan vendor:publish --provider="Laravel\Fortify\FortifyServiceProvider"
```

Por último realizaremos la ejecución de las migraciones.

```
php artisan migrate
```

Registrar el service provider de Fortify

Entre los archivos que hemos agregado en la instalación de Fortify encontraremos un *ServiceProvider* que tenemos que registrar. Se trata de la clase `App\Providers\FortifyServiceProvider` que tenemos que agregar al array de providers de `config/app.php`.

En el array de providers añadimos la siguiente línea:

```
App\Providers\FortifyServiceProvider::class,
```

Siguientes pasos con Fortify

Hasta aquí hemos instalado Laravel Fortify y tenemos ya con nosotros la serie de rutas de aplicación necesarias para trabajar con toda la parte de autenticación de usuarios en nuestro proyecto.

Para verificar que hemos conseguido el objetivo perseguido con la instalación de Laravel Fortify podemos **comprobar la lista de rutas de aplicación** con este comando:

```
php artisan route:list
```

```
midesweb@MacBook-Pro  ~/Sites/time-control/tc-back  sail php artisan route:list

  GET|HEAD  /
  POST      _ignition/execute-solution ..... ignition.executeSolution › Spatie\LaravelIgnition › ExecuteSolutionController
  GET|HEAD  _ignition/health-check ..... ignition.healthCheck › Spatie\LaravelIgnition › HealthCheckController
  POST      _ignition/update-config ..... ignition.updateConfig › Spatie\LaravelIgnition › UpdateConfigController
  GET|HEAD  api/user
  GET|HEAD  forgot-password ..... password.request › Laravel\Fortify › PasswordResetLinkController@create
  POST      forgot-password ..... password.email › Laravel\Fortify › PasswordResetLinkController@store
  GET|HEAD  login ..... login › Laravel\Fortify › AuthenticatedSessionController@create
  POST      login ..... Laravel\Fortify › AuthenticatedSessionController@store
  POST      logout ..... logout › Laravel\Fortify › AuthenticatedSessionController@destroy
  GET|HEAD  register ..... register › Laravel\Fortify › RegisteredUserController@create
  POST      register ..... Laravel\Fortify › RegisteredUserController@store
  POST      reset-password ..... password.update › Laravel\Fortify › NewPasswordController@store
  GET|HEAD  reset-password/{token} ..... password.reset › Laravel\Fortify › NewPasswordController@create
  GET|HEAD  sanctum/csrf-cookie ..... sanctum.csrf-cookie › Laravel\Sanctum › CsrfCookieController@show
  GET|HEAD  two-factor-challenge ..... two-factor.login › Laravel\Fortify › TwoFactorAuthenticatedSessionController@create
  POST      two-factor-challenge ..... Laravel\Fortify › TwoFactorAuthenticatedSessionController@store
  GET|HEAD  user/confirm-password ..... Laravel\Fortify › ConfirmablePasswordController@show
  POST      user/confirm-password ..... password.confirm › Laravel\Fortify › ConfirmablePasswordController@store
  GET|HEAD  user/confirmed-password-status ..... password.confirmation › Laravel\Fortify › ConfirmedPasswordStatusController@show
  POST      user/confirmed-two-factor-authentication ..... two-factor.confirm › Laravel\Fortify › ConfirmedTwoFactorAuthenticationController@store
  PUT       user/password ..... user-password.update › Laravel\Fortify › PasswordController@update
  PUT       user/profile-information ..... user-profile-information.update › Laravel\Fortify › ProfileInformationController@update
  POST      user/two-factor-authentication ..... two-factor.enable › Laravel\Fortify › TwoFactorAuthenticationController@store
  DELETE    user/two-factor-authentication ..... two-factor.disable › Laravel\Fortify › TwoFactorAuthenticationController@destroy
  GET|HEAD  user/two-factor-qr-code ..... two-factor.qr-code › Laravel\Fortify › TwoFactorQrCodeController@show
  GET|HEAD  user/two-factor-recovery-codes ..... two-factor.recovery-codes › Laravel\Fortify › RecoveryCodeController@index
  POST      user/two-factor-recovery-codes ..... Laravel\Fortify › RecoveryCodeController@store
  GET|HEAD  user/two-factor-secret-key ..... two-factor.secret-key › Laravel\Fortify › TwoFactorSecretKeyController@show

                                                                                                  Showing [29] routes
```

En el siguiente artículo vamos a abordar algunas configuraciones para Fortify que creo que son útiles, y en muchos casos necesarias, para los proyectos de API realizados mediante Laravel.

Este artículo es obra de *Miguel Angel Alvarez*

Fue publicado / actualizado en *13/12/2023*

Disponible online en *https://desarrolloweb.com/articulos/laravel-fortify*

Configuración de Laravel Fortify

Cómo configurar Laravel Fortify y personalizar su comportamiento para que funcione de la manera adecuada para el desarrollo de un API. Resolver problemas comunes de configuración de Fortify.

En los anteriores artículos del Manual de desarrollo de un API con Laravel Sanctum y Fortify hemos avanzado bastante en la realización del sistema de autenticación. Después de haber conocido e instalado en el artículo anterior Laravel Fortify, ahora tenemos que realizar algunas tareas de **adaptación de Fortify** según las necesidades de nuestro proyecto.

> Las necesidades de nuestro proyecto vienen marcadas porque vamos a **usar Fortify en el contexto de un API**. Recuerda que Fortify no necesita necesariamente funcionar en un API, por lo que hay algunas configuraciones de casa que están más pensadas para sitios web. Tendremos que cambiarlas como verás a continuación y, de paso, nos dará pie a aprender algunas cosas más sobre este package de Laravel.

Configurar las funcionalidades de Fortify

En este paso vamos a editar el **archivo de configuración del propio Fortify**: `config/fortify.php`. En él encontramos un array llamado "`features`", el cual podemos **activar o desactivar** aquellas que nos interesen o no.

Si queremos las funcionalidades básicas de un sistema de autenticación, podríamos dejar algo así.

```
'features' => [
        Features::registration(),
        Features::resetPasswords(),
        Features::emailVerification(),
        // Features::updateProfileInformation(),
        // Features::updatePasswords(),
        // Features::twoFactorAuthentication([
        //     'confirm' => true,
        //     'confirmPassword' => true,
        //     // 'window' => 0,
        // ]),
],
```

No te preocupes porque hayamos dejado de lado algunas de las funcionalidades ofrecidas por Fortfy. Las que no hemos activado todavía se podrían activar más adelante si fuera necesario.

Desactivar las vistas

Para el uso que vamos a hacer de nuestro backend necesitamos **desactivar las vistas de la aplicación para lo que respecta a Fortify**. Esto lo realizamos también mediante el archivo de configuración de Fortify. Dentro del archivo `config/fortify.php` también buscamos el siguiente valor:

```
'views' => true,
```

Y lo cambiamos por

```
'views' => false,
```

En este punto es relevante comentar que la vista del "*reset de la clave*" si que la necesitaremos cuando activemos esa funcionalidad. La tendremos que activar manualmente en los archivos de rutas de nuestro proyecto backend con Laravel.

Este punto requiere más explicaciones para dejar claro cómo activar estas vistas. De todos modos, vamos a aplazar estas explicaciones más adelante cuando abordemos el flujo para recordar la clave.

Comportamiento de redirección si estás autenticado

Hay otra cosa que resulta molesta del comportamiento de Fortify, en el caso de querer usar el sistema como un servicio web (API). Se trata de la redirección a la ruta "/home" que se realiza cuando el usuario está autenticado e intenta hacer de nuevo un login.

Este comportamiento se debe a un middleware llamado `RedirectIfAuthenticated`, que está en la ruta `app/Http/Middleware/RedirectIfAuthenticated.php`.

Su código lo vamos a cambiar para que incluya este otro comportamiento, que permite detectar si estamos **accediendo mediante Ajax a la ruta de login** y, en caso que sea así, se envíe una respuesta en formato JSON.

El código que tenemos que añadir es este:

```
if ($request->expectsJson()) {
            return response()->json(['error' => 'Ya estás autenticado.'], 200);
```

```
}
```

> El método `$request->expectsJson()` devuelve `true` cuando se esté invocando el login desde una solicitud Ajax.

Esa comprobación la tenemos que poner dentro del bucle foreach, donde ahora el código quedará de esta manera.

```
foreach ($guards as $guard) {
       if (Auth::guard($guard)->check()) {
           if ($request->expectsJson()) {
             return response()->json(['error' => 'Ya estás autenticado.'], 200);
           }
           return redirect(RouteServiceProvider::HOME);
      }
}
```

Básicamente, lo que hemos hecho es que se **envíe un mensaje indicando que ya tienes el usuario autenticado**, por lo que no necesitarías acceder de nuevo a la ruta de login.

> Este comportamiento lo podrías redefinir para que tu API devuelva la respuesta que tú necesites, con el código HTTP que te resulte adecuado.

Comportamiento si no estás autenticado

El middleware `Authenticate`, en la ruta `app/Http/Middleware/Authenticate.php` también sería interesante cambiarlo. Básicamente este middleware realiza el trabajo de **enviar al usuario a la ruta donde debería hacer su autenticación**, para cada ruta que configuremos que requiera estar logueado.

Vamos a cambiarlo para que **redirija a la ruta del frontend donde se hace el login**.

En principio tenemos este código.

```
protected function redirectTo(Request $request): ?string
{
    return $request->expectsJson() ? null : route('login');
}
```

Podemos cambiarlo por este otro.

```
protected function redirectTo(Request $request): ?string
{
    if (!$request->expectsJson()) {
        return url(env('FRONTEND_URL') . '/login');
    }
```

```
    return null;
}
```

Si te fijas, estamos usando una variable de entorno nueva, por lo que necesitarás crearla en el archivo `.env`. La variable de entorno `FRONTEND_URL` deberá tener la ruta donde tienes la aplicación frontend.

El valor de la variable de entorno podría ser algo como esto:

```
FRONTEND_URL=http://localhost:8000
```

Problemas comunes de la configuración de Laravel Fortify

Ahora voy a señalar algunos problemas comunes que he encontrado a la hora de dar estos primeros pasos en la configuración de Laravel Fortify, con sus soluciones.

Solucionar 401 (Unauthorized) en Fortify

Durante el proceso de creación de tu sistema de login puedes apreciar un mensaje *401 (Unauthorized)* cuando intentas **acceder a una ruta para la cual necesitas estar autenticado**.

Esta no debería ser la respuesta que el API te debería entregar, en el caso que ya hayas hecho login. Si has verificado que tu sesión está activa, porque hayas hecho un registro o login satisfactorio y aún así sigues recibiendo este mensaje puede deberse a dos motivos:

1. Qué el dominio de tu frontend, desde donde estás haciendo la solicitud no esté entre los que se permite el inicio de sesión con variables de sesión. Entonces debes de asegurarte que la variable de entorno `SANCTUM_STATEFUL_DOMAINS`contiene un valor correcto. Atención en este punto poque debemos colocar el valor sin `http://`, que es uno de los errores típicos que pueden suceder. El valor es simplemente el nombre del dominio y el puerto, si es que estás usando un puerto personalizado. Por ejemplo: `SANCTUM_STATEFUL_DOMAINS=localhost:8000`
2. Que no se estén creando las variables de sesión en el backend de manera correcta. En este caso debes asegurarte que no tienes la variable de sesión `SESSION_DOMAIN` en tu archivo `.env`. O, en el caso de tenerla, que contenga el dominio de tu backend, sin `http://` o `https://`. Por ejemplo para desarrollo sería algo así `SESSION_DOMAIN=localhost`.

Solucionar has been blocked by CORS policy

Otro problema típico que te puede ocurrir es que las **solicitudes se bloqueen por la política de CORS**. En estos casos el acceso a las rutas te pueden entregar mensaje como este:

```
Access to XMLHttpRequest at 'http://localhost/sanctum/csrf-cookie' from origin 'http://localhost:8000' has been
blocked by CORS policy: Response to preflight request doesn't pass access control check: No 'Access-Control-Allow-
Origin' header is present on the requested resource.
```

Para solucionar este problema verifica:

- Que en `config/cors.php` tienes la ruta a la que quieres acceder en el array paths.
- Que tienes el valor `allowed_origins` en `config/cors.php` asignado a `['*']` o en su defecto a los dominios donde está permitido el uso del API.

El valor de `allowed_origins` podría estar definido, por ejemplo, de la siguiente manera:

```
'allowed_origins' => ['http://localhost:8000', 'https://escuela.it']
```

.

> Podrías tener una variable de entorno para definir la URL de tu SPA y usarla en este punto.

Conclusión a la configuración básica de Laravel Fortify

Hasta aquí hemos podido terminar la configuración de nuestro backend para que pueda realizarse la autenticación de usuarios, usando los packages oficiales de Laravel Sanctum y Laravel Fortify.

Por supuesto, estarás deseando probar tu backend y realizar las tareas de autenticación con él, para verificar que todo está funcionando correctamente. Para realizar esta tarea no podríamos utilizar directamente con el navegador, porque las solicitudes deben realizarse mediante Ajax, además de enviar los datos correctos al backend para realizar el registro, autenticación y autorización. Por ello en el próximo artículo vamos a detenernos para mostrar un medio de ejercitar nuestra API y mandar las solicitudes necesarias para ponerla en marcha.

> Este artículo es obra de *Miguel Angel Alvarez*
>
> Fue publicado / actualizado en *27/03/2023*
>
> Disponible online en *https://desarrolloweb.com/articulos/configuracion-laravel-fortify*

Proyecto frontend SPA

En estos artículos vamos a tocar la parte del frontend que necesitamos para poder usar el proyecto de API Laravel. Desarrollar un proyecto frontend con autenticación no es tarea inmediata, pero podemos usar varios starter kits. Estos proyectos nos permitirán construir una SPA en pocos minutos preparada para usar el backend que hemos desarrollado en los anteriores artículos. Existen alternativas oficiales, creadas y mantenidas del propio equipo de Laravel, como alternativas no oficiales desarrollados por la comunidad. En cualquier caso, usar una u otra alternativa es muy similar, por lo que básicamente la elección dependerá del stack de tecnologías frontend que prefieras tú para tu proyecto.

Montar una aplicación SPA con autenticación vía API con el starter kit oficial de Laravel

Cómo construir una aplicación frontend moderna, tipo SPA, en pocos minutos con los starter kits oficiales de Laravel. Cómo instalar y arrancar el proyecto Laravel Breeze - Next.js Edition.

En los artículos anteriores del Manual del desarrollo de un API con Laravel **hemos visto cómo crear una aplicación backend que ofrece servicios de autenticación y registro de usuarios**, entre otras cosas. Tenemos por tanto un backend ya listo para experimentar su funcionamiento, pero no podemos usar el navegador tal cual, escribiendo la URL del servicio web, ya que necesitamos enviarle los datos de autenticación o registro.

Entonces, **para poder usar este backend** lo que vamos a hacer es montar una **aplicación frontend del tipo SPA**, con la que podremos probar si todo está funcionando correctamente. Sin embargo, montar una SPA no es una tarea trivial y necesitaríamos unas cuantas horas para tenerlo todo listo. Por tanto, lugar de empezar todo desde cero, vamos a utilizar un *starter kit* que nos

proporcione ya la base de la aplicación lista, a partir de la cual luego podríamos personalizar nuestra aplicación para aportar la funcionalidad específica que necesitemos.

En este artículo vamos a poner un poco de orden en todas las alternativas que se nos abren ahora que queremos meter mano en nuestro frontend. Veremos qué tipos de starter kits ofrece Laravel de forma oficial, analizando sus diferencias y veremos cómo instalar, configurar y arrancar un proyecto de SPA ya realizado que te puede servir como punto de inicio para crear tu aplicación frontend que consuma los servicios de autenticación del backen que hemos desarrollado hasta este punto del manual.

Starter Kits oficales de Laravel

Con todas las alternativas de starter kits de aplicaciones frontend que vamos a ver en este artículo podrías montar tu cliente de API, para backends que se hayan realizado de dos modos distintos:

- Mediante **Laravel Sanctum y Laravel Fortify**, tal como hemos explicado en este manual
- Mediante **Laravel Breeze con el scaffolding de API**

> Ambos tipos de proyectos backend realizados con Laravel usando packages distintos permiten una arquitectura muy similar, al menos en las funcionalidades más básicas como el login y el registro. La alternativa de Laravel Sanctum y Laravel Fortify tiene la particularidad que permite además el uso de tokens de API, además de cookies y sesiones, con lo que ese backend lo podrías usar también para aplicaciones móviles o aplicaciones de escritorio.

Notas sobre los starter Kits oficiales de Laravel

Laravel tiene diversos **starter kits oficiales** desde los que podemos partir para el desarrollo de una aplicación web. Son los siguientes:

- Laravel Breeze
- Laravel Jetstream

Primero queremos aclarar que estos proyectos son ideales para construir un sitio web con Laravel, pues te ofrecen toda la infraestructura ya lista para algunas de las funcionalidades más comunes como el inicio de sesión, registro y asuntos extra como recordar la clave, verificar emails, etc. Pero ambos proyectos son para sitios web tradicionales, multipágina, no para una aplicación frontend moderna con el modelo de SPA. Nuestra idea en este caso es tener un proyecto de SPA (la parte frontend) que sea completamente independiente de la parte del backend.

Si lo que tienes que hacer es un sitio web tradicional, te recomendamos usar Laravel Breeze para un inicio sencillo de tu aplicación. Por otra parte recomensamos Laravel Jetstream para sitios web más sofisticados, siendo que además Jetstream tiene funcionalidades extra como la autenticación a dos pasos.

> Jetstream usa por debajo Laravel Fortify, por lo que si estás usando Fortify tendrías también la posibilidad de hacer autenticación a dos pasos, aunque no uses Jetstream.

Si lo que quieres es tener un frontend completamente aislado del backend, entonces te vendrán bien los starter kits basados en stacks frontend que vamos a explicar en este artículo.

Starter Kit oficial de Laravel para una SPA: Laravel Breeze - Next.js Edition

Vamos a comenzar explicando cómo usar el starter kit oficial de Laravel basado en un framework frontend: Laravel Breeze - Next.js Edition

Este es un proyecto realizado con el framework NextJS, que está basado en React. Se trata de una implementación de Laravel Breeze, solo que no está desarrollada con vistas Blade en una aplicación Laravel, sino que está realizada mediante un proyecto frontend puro.

Este proyecto está pensado para servir como starter kit de una aplicación frontend SPA que se comunicará con un backend de API realizado con Laravel, para la autenticación por medio de sesiones y cookies.

Para usar este proyecto tienes que:

- Clonar el repositorio indicado
- Instalar las dependencias con npm

El comando de clonado lo harás con Git y la URL del repositorio. Puedes ver este artículo si tienes dudas sobre cómo clonar repositorios Git.

Una vez clonado, las dependencias las instalas con npm, para lo que es necesario que tengas NodeJS instalado en tu ordenador. Una vez instalado Node, las dependencias las instalas con este comando, lanzado desde la carpeta del proyecto clonado.

```
npm install
```

Posteriormente debes arrancar la aplicación NextJS. Esto lo consigues con uno de los scripts npm que tienes en el package.json. Lo ejecutas con este comando de terminal, situado en la carpeta del proyecto.

```
npm run dev
```

Además, es posible que necesites un cambio en la URL donde está el backend. Este proyecto espera que esté en `localhost:8000` y si no fuera así deberías cambiarlo con estos pasos:

- Entrar en la carpeta "src" del proyecto NextJS
- Hacer una copia del archivo .env.example y llamarla .env

- Cambiar la URL del backend como variable de entorno

Por ejemplo, si estamos trabajando con Sail en el backend tendríamos que quitar el puerto en la URL de localhost.

```
NEXT_PUBLIC_BACKEND_URL=http://localhost
```

Conclusión

Si usas este proyecto frontend para implementar una SPA podrás tener un proyecto de aplicación frontend moderna en pocos minutos, con el flujo de autenticación ya desarrollado, lo que te quitará mucho trabajo. La aplicación debería comunicarse con el backend que hemos desarrollado en los pasos anteriores del manual del desarrollo de un API con Laravel Sanctum y Fortify.

Sin embargo, no todo el mundo conoce este modelo de proyecto basado en NextJS o prefiere otros stack de desarrollo de aplicaciones frontend. Existen muchas alternativas muy útiles en el mundo de la web, basadas en otras librerías como Vue o directamente en el estándar Web Components. En el próximo artículo abordaremos algunas alternativas no oficiales que puedes encontar para el inicio de una SPA con autenticación que funciona con un backend Laravel.

Este artículo es obra de *Miguel Angel Alvarez*
Fue publicado / actualizado en *10/05/2023*
Disponible online en *https://desarrolloweb.com/articulos/spa-api-laravel-starter-kit*

Alternativas no oficiales de starter kits frontend para API Laravel

Conoce una selección de proyectos frontend para consumir un API desarrollada con Laravel y Sanctum, alternativas al starter kit oficial de Laravel y con tecnologías diversas: Nuxt, Web Components y Lit, Electron.

En el artículo anterior del Manual de desarrollo de un API con Laravel comenzamos a hablar de la parte del frontend. Si bien es cierto que al desarrollar con Laravel nos ocuparemos de la parte del backend, ellos sí te ofrecen un starter kit basado en Next.JS que nos permite ahorrar muchas horas de desarrollo, al darte la base de código necesaria para comunicar con Laravel en todo lo que respecta a los procesos de login de usuarios y registro.

Sin embrargo, puede que no a todo el mundo le guste Next.JS y React, por lo que en este artículo queremos ofrecer referencias y consejos de alternativas para trabajar con otros starter kits que ocupan stacks de tecnologías frontend distintas.

Alternativa no oficial de starter Kit basada en Vue

Comenzamos presentando una alternativa de proyecto con Vue para realizar la autenticación en backends creados con Laravel Sanctum te sugerimos consultar este proyecto: https://github.com/filippetrovicAT/breeze-nuxt

En este caso tenemos una alternativa basada en Nuxt, muy inspirada en el funcionamiento del proyecto oficial de Laravel, que estaba basado en NextJS. Se trata de ofrecer lo mismo que ofrece Laravel Breeze pero en una aplicación completa hecha en un proyecto frontend del estilo de SPA.

Comunica de manera sencilla con el API realizada con Laravel Sanctum, de modo que en pocos minutos tendrías la estructura de aplicación frontend enganchada con el API de Laravel y capaz de realizar todo el proceso de registro y login de usuarios, autorización, etc.

Alternativa de SPA basada en Web Components y Lit

Además quiero presentar aquí una alternativa más cercana al Javascript nativo, ya que usa el estándar de Web Components y tiene menos dependencias de tecnologías frontend, apenas con la librería Lit que nos permite desarrollar Custom Elements que funcionan en cualquier navegador y framework.

Esta alternativa además tiene unos grados de acabado un poco más sofisticados, ya que implementa también el flujo para la verificación del email de los usuarios, que no está disponible en otros starter kits.

Este proyecto también está altamente inspirado en el repositorio oficial de Laravel, aunque con un stack de tecnologías pegado al propio Javascript, lo que permite tener menos código Javascript de terceros. Con ello conseguimos mayor rendimiento, menor peso de aplicación y a la vez se reduce el consumo de batería en dispositivos, ya que que no se usa un framework, solamente la micro-librería Lit, que tiene un peso menor de 6 Kb (Gzipped).

Como herramienta frontend se usa además Vite, que es una maravilla para la experiencia de desarrollo, ya que es sencillo y ligero.

El repositorio de este proyecto es https://github.com/Polydile/dile-app

Para usar este proyecto realizamos los siguientes pasos:

Clonar el repositorio

```
git clone https://github.com/Polydile/dile-app.git
```

Ahora nos metemos en la carpeta del proyecto que acabamos de clonar y luego tenemos que ejecutar npm (o cualquier otro gestor) para instalar las dependencias:

```
npm install
```

Ahora, para arrancar el servidor de desarrollo que nos ofrece Vite, debemos ejecutar el script `dev`:

```
npm run dev
```

Este proyecto espera que tu backend esté en `http://localhost`. Si esa no es la URL de tu backend, entonces necesitas crear una variable de entorno en el archivo `.env` llamada `VITE_BACKEND_URL` con el valor de la URL donde se encuentre tu backend. Podría ser algo como esto:

```
VITE_BACKEND_URL=http://localhost:8000
```

Existe un archivo llamado `.env_example` que tiene esa variable de configuración. Puedes copiar ese archivo en la misma carpeta donde se encuentra, con el nombre `.env` y cambiar el valor de la variable de entorno por el que tú necesites.

Starter Kit para API Laravel con Electron

Antes de acabar hay otro proyecto que querría comentar basado en Electron.js que también nos puede ofrecer una base de código interesante, en este caso para el desarrollo de una aplicación de escritorio que usa un backend de API desarrollado con Laravel.

Este proyecto lo podemos encontrar en https://github.com/Polydile/dile-app-electron y está bastante maduro, aunque sigue siendo un proyecto en vías de desarrollo.

La diferencia con los proyectos anteriores es que este basado en Electron está pensado para trabajar con autenticación basada en tokens, en lugar de autenticación basada en cookies de sesión.

> Como hemos dicho ya, Laravel Sanctum permite trabajar tanto con cookies de sesión para aplicaciones statefull y con tokens JWT para el modelo de API Stateless. El trabajo con tokens es el adecuado para aplicaciones móviles o proyectos donde pensamos que va a ser esencial asuntos como la escalabilidad.

Este proyecto de starter kit tiene una base de código bastante completa, aunque le falta todavía la documentación. También usa Lit como librería.

Conclusión

Con este artículo has podido encontrar diversas alternativas de proyectos frontend preparados para ejercitar un backend de API con Laravel Sanctum basado en sesiones e incluso para proyectos Laravel Sanctum basados en tokens JWT. Los dos primeros tambien funcionarían con el scaffolding de Laravel Breeze para API.

Son alternativas con diversos stacks de tecnologías frontend, con frameworks como Nuxt o ElectronJS y con librerías como Lit.

Si estamos trabajando con el scaffolding de Laravel Breeze para API yo prefiero la opción basada en Web Components y Lit, por ser la alternativa más cercana al Javascript Vanilla. Al trabajar con el estándar de Web Components me siento muy cómodo porque es un estándar al que esperamos vida larga, como todos los estándares. Además esa opción es la que ofrece mayor cantidad de funcionalidades, ya que implementa vistas para recordar la clave y verificar el email de los usuarios.

Además, el proyecto basado en Electron es un buen ejemplo sobre cómo se tendría que desarrollar un proyecto frontend donde tengamos que autenticarnos con un API basada en tokens en lugar de sesiones.

Este artículo es obra de *Miguel Angel Alvarez*

Fue publicado / actualizado en *27/10/2023*

Disponible online en *https://desarrolloweb.com/articulos/alternativas-starter-kits-frontend-api-laravel*

Actividades opcionales para completar el flujo de registro de usuarios

Ahora vamos a ver algunas prácticas opcionales que pueden mejorar el flujo de registro y autenticación de usuarios en proyectos de API desarrollados con Laravel.

Flujo para la verificación del email del usuario en Laravel Fortify en un API

Configuramos un API backend desarrollado con Laravel que será consumido desde una aplicación frontend SPA para que pueda verificar el email de los usuarios registrados.

Uno de los servicios básicos que podemos implementar fácilmente en nuestra aplicación SPA es la **verificación del correo de los usuarios**. Esta es una funcionalidad muy habitual en las aplicaciones con consiste en enviar un email al correo de cada usuario según se autentica. En ese correo hay un enlace para verificar que el usuario registrado posee el correo que ha usado para registrarse. Seguro que sabes a qué nos referimos porque habrás pasado por ese proceso en muchas ocasiones.

La verdad es que Laravel ya tiene todo el proceso realizado cuando instalamos un starter kit como Breeze, que nos incluye todo el proceso de registro completo, tanto la parte del backend como del frontend. Si hemos optado por realizar un API y apoyarnos en el package de Laravel Fortify también tenemos implementada toda la operativa de la parte del backend, aunque tenemos que realizar algunas alteraciones en el código base de la aplicación Laravel.

Damos por hecho que nuestra aplicación backend está pensada para implementar un API y que has usado Sanctum y Fortify para toda la parte del proceso de login y registro. Si no es así, tendrías que comenzar creando tu aplicación y configurando los packages necesarios, tal como se explicó en el Manual de desarrollo de API con Laravel Sanctum y Fortify.

Para integrar el proceso de verificación del email con tu aplicación frontend tendrás que seguir los siguientes pasos.

Creamos una variable de entorno FRONTEND_URL

Si no la has creado todavía, en el archivo de variables de entorno `.env` vamos a crear una nueva entrada llamada "`FRONTEND_URL`" a la que asignaremos la URL del frontend que se conectará con nuestro API backend.

```
FRONTEND_URL=http://localhost:8000
```

Esta URL la necesitamos conocer en el backend porque **nos permitirá hacer las correspondientes redirecciones al frontend desde las rutas de verificación del email**.

Activar la funcionalidad de verificación de emails

Debemos asegurarnos que la funcionalidad de verificación de los emails registrados está activada. Esto lo conseguimos en el archivo de configuración de Fortify, en `config/fortify.php`.

Allí debes localizar el array de `features` y descomentar la línea de "`emailVerification`".

```
'features' => [
        Features::registration(),
        Features::resetPasswords(),
        Features::emailVerification(),
        // …
]
```

Usar la variable de entorno de FRONTEND_URL en la configuración de Fortify

También en el archivo de configuración de Fortify `config/fortify.php` tenemos que alterar el valor de la variable '`home`', usando la URL definida en la variable de entorno.

```
'home' => env('FRONTEND_URL'),
```

En el caso que, al verificar el email, quieras llevar los usuarios a otra ruta, no necesariamente la home de tu aplicación frontend, podrías dejar una configuración parecida a la siguiente:

```
'home' => env('FRONTEND_URL') . '/dashboard',
```

Asegurarnos que CORS está activo para la ruta de verificación del email

En la **configuración de CORS** tenemos que **activar una ruta más para consultar desde el frontend**. Básicamente es la ruta desde la que podemos pedir que nos reenvíen el email de verificación del usuario.

En `config/cors.php` debemos asegurarnos de que cors está activado para la ruta nueva que quizás no tengamos todavía. Se trata de la ruta `/email/verification-notification`.

El archivo de cors.php tendría una configuración de `paths` con un código similar a este:

```
'paths' => [
        'api/*',
        'sanctum/csrf-cookie',
        'login',
        'register',
        'logout',
        '/email/verification-notification',
],
```

> Podría ser el caso que tuvieses un array de `paths` con el valor `'*'`, con un código como el que sigue en la configuración de rutas:
>
> ```
> 'paths' => ['*'],
> ```
>
> En este caso el asterisco actúa de comodín e incluye a todas las rutas de la aplicación. Quiere decir que todas las rutas tienen activado CORS, por lo que también lo estará la ruta `/email/verification-notification`.

Configurar el modelo de User

En el modelo de usuario que está en el fichero `app/Models/User.php` tenemos que hacer una mínima configuración para que se envíen los correos de verificación al registrar a los usuarios. Esta modificación consiste en implementar la interfaz `MustVerifyEmail`.

También debemos usar el trait `Notifiable`. El código relevante del modelo `User` sería el siguiente.

```
class User extends Authenticatable implements MustVerifyEmail
{
    use HasApiTokens, HasFactory, Notifiable;
```

Gracias a estos cambios, todos los usuarios que se registren recibirán automáticamente el email de verificación de su correo electrónico. Además, dispondremos en el backend de una ruta para solicitar de nuevo el email de verificación.

La ruta para verificar el email (la que te envía al pulsar el enlace de verificación del email) **la genera Laravel en el backend gracias al package Fortify**. Así que no necesitamos hacer

nada en particular para recibir las verificaciones. Simplemente gracias a la configuración de Fortify podemos definir dónde se reenviará al usuario al verificar su cuenta.

Conclusión

Con estos pasos hemos conseguido activar todos los requerimientos del backend para poder verificar los emails de los usuarios. Por supuesto, esto solamente ha sido una parte de tu trabajo, ya que ahora en el frontend deberías programar todas las vistas de tu aplicación y las comprobaciones necesarias para saber si tus usuarios están o no verificados.

De todos modos, en alguno de los starter kit que vimos en el artículo anterior, al menos en el basado en Web Components y Lit, se incluyen todas las vistas necesarias para soportar la verificación del email. Además se muestra cómo realizar las comprobaciones necesarias para saber si un usuario tiene o no el email verificado y mostrar los correspondientes mensajes para verificación.

Este artículo es obra de *Miguel Angel Alvarez*

Fue publicado / actualizado en *14/11/2023*

Disponible online en *https://desarrolloweb.com/articulos/verificacion-email-usuario-laravel-fortify-api*

Modificaciones en el backend para recordar la clave de un usuario con Laravel Fortify

Cómo adaptar Laravel Fortify para que pueda realizarse la funcionalidad de recordar una clave en un API que usamos desde un frontend separado, activando CORS y redireccionando correctamente a las rutas del frontend.

Antes de empezar, voy a dar un poco de contexto a esta serie de artículos, donde estamos abordando el desarrollo de una aplicación distribuida, con Backend y Frontend desacoplados. En el backend estamos usando Laravel con Sanctum y Fortify. Más información de la serie en el Manual del desarrollo de API con Laravel.

En este artículo vamos a encargarnos de describir una posibilidad para implementar en nuestro backend muy necesaria. Se trata de **realizar la operativa de recordar la clave**, fundamental para darle soporte al usuario en el caso de que se la haya olvidado su contraseña y no pueda hacer inicio de sesión.

Básicamente nuestro trabajo será **proporcionar una ruta desde donde se solicite el envío de la notificación al usuario con el link para crear una nueva clave**. Además, en el frontend colocaremos el **formulario para generar una nueva clave**.

Para comenzar el proceso en el frontend tenemos que crear un formulario que envíe por POST el email del usuario sobre el que queremos operar, aquel sobre el que se desea crear una nueva clave. Este formulario se enviará a la ruta `/forgot-password`.

Fortify tiene ya todo lo necesario para componer el email que debe permitir al usuario recordar la clave. Solo tenemos que asegurarnos de los siguientes requisitos que vamos a describir.

En el artículo **Modificaciones en el backend para recordar la clave de un usuario con Laravel Fortify** encuentras los siguientes apartados de interés.

- Activar la funcionalidad de reset password
- Activar Cors para la ruta 'forgot-password',
- Problema de generación de la ruta para resetear la clave
- Redefinir la función que genera el enlace para resetear la clave
- Enviar los datos del la nueva clave

Activar la funcionalidad de reset password

Fortify debe tener activada la funcionalidad de resetear el password. Esto se hace en el archivo `config/fortify.php`

```
'features' => [
        // varias funcionalidades…
        Features::resetPasswords(),
]
```

Simplemente debemos verificar que la línea de la funcionalidad de `resetPasswords` está activada.

Activar Cors para la ruta 'forgot-password',

En el archivo de configuración de Cors necesitaremos asegurarnos también de que esté activa la ruta de '`forgot-password`'. Esto lo conseguiremos activando la ruta en el array de paths, si es que no la teníamos ya registrada.

```
'paths' => [
    'api/*',
    'sanctum/csrf-cookie',
    'login',
    'register',
    'logout',
```

```
    '/email/verification-notification',
    'forgot-password',
    'reset-password',
],
```

Además observarás que también tenemos '`reset-password`' entre las rutas con Cors. Esta ruta la necesitaremos también un poco más adelante.

Problema de generación de la ruta para resetear la clave

Según nos advierten en la documentación de Laravel Fortify, cuando se genera el enlace para que el usuario pueda resetear la clave, incluyendo el token necesario, se usa como referencia una ruta del backend donde tiene la vista con el formulario para que el usuario pueda escribir la nueva clave.

Nuestro problema es que las rutas del backend están desactivadas, pues este backend no tiene vistas, ya que estamos usándolo solo como un API. Esto ocurre porque en la configuración de Fortify marcamos la opción:

```
'views' => false,
```

Esto produjo que no se registren las rutas de las vistas y por lo tanto no se podrá generar el enlace para resetear la clave.

Al intentar componer el email con el enlace para resetear la clave el backend lanzará un error 500 con el siguiente mensaje: *Route [password.reset] not defined*.

Para solucionarlo podríamos crear una ruta en nuestro backend con el "name" "password.reset". En ese caso el backend tendría que generar además un formulario para reescribir la clave, pero llegaríamos a un callejón sin salida porque este backend está manipulado para funcionar solamente como un API, sin páginas visibles.

Lo mejor es cambiar la funcionalidad de Laravel para crear el enlace con el token. Esto es algo que está explicado en la documentación en la sección Reset link customization. De todos modos te lo vamos a explicar aquí.

Redefinir la función que genera el enlace para resetear la clave

Entendido el problema anterior, nuestra solución preferida consiste en simplemente cambiar el enlace que llevaría a la ruta desde la que se puede resetear la clave. En vez de llevar al usuario a la ruta del backend lo llevaremos a una ruta en nuestra aplicación frontend.

Para ello tenemos que hacer unas modificaciones en el archivo `AuthServiceProvider.php` que tenemos en la ruta `app/Providers/AuthServiceProvider.php`.

En ese archivo tenemos que comenzar haciendo el uso de una clase llamada `ResetPassword` que vamos a necesitar en su namespace:

```
use Illuminate\Auth\Notifications\ResetPassword;
```

Luego tenemos que invocar al método estático `createUrlUsing()` de `ResetPassword`. En ese método estático le vamos a enviar como parámetro el código de la función que se encargará de crear el enlace de resetear la clave.

Esa función recibe como parámetros el usuario y el token. Debe devolver la cadena con la ruta donde ese usuario debe cambiar su clave. Podría ser algo como lo que sigue:

```
public function boot()
{
    $this->registerPolicies();

    ResetPassword::createUrlUsing(function ($user, string $token) {
        return env('FRONTEND_URL') . '/reset-password?token='.$token;
    });
}
```

También podríamos enviar el email de ese usuario si lo deseamos:

```
public function boot()
{
    $this->registerPolicies();

    ResetPassword::createUrlUsing(function ($user, string $token) {
        return env('FRONTEND_URL') . '/reset-password?token='.$token . '&email=' . $user->email;
    });
}
```

El email no es necesario enviarlo realmente, dado que el usuario lo puede introducir a mano, pero si lo enviamos mediante el querystring podríamos poblar el campo de formulario y evitaríamos al usuario tener que introducirlo.

Enviar los datos del la nueva clave

Ahora ya solamente quedaría componer en el frontend un formulario para enviar los datos de la nueva clave.

Ese formulario necesitaría enviar al backend el token de vuelta, junto con el email del usuario y la clave, así como la conformación de la clave. La ruta donde se espera recibir ese formulario es `/reset-password` y el método POST.

Esa parte la veremos cuando lleguemos a componer nuestro frontend.

Este artículo es obra de *Miguel Angel Alvarez*
Fue publicado / actualizado en *09/01/2024*

Disponible online en *https://desarrolloweb.com/articulos/backend-recordar-clave-api-laravel-fortify*

Trabajo con API en el modelo stateless

Ahora vamos a abordar todo lo que respecta al trabajo con tokens de API para la autenticación de usuarios usando el modelo stateless, que es el más habitual en las API REST.

Autenticación con tokens para API usando Sanctum en Laravel

Cómo trabajar con Sanctum para el desarrollo de la autenticación de usuarios en API REST con el modelo stateless, en Laravel.

En este artículo vamos a cambiar un poco de tercio, en relación a todo lo que veníamos abordando en el Manual del Desarrollo de APIs con Laravel. Hasta ahora hemos trabajado siempre en el modelo de trabajo basado en sesiones (stateful), pero es solo una de las posibilidades de Laravel Sanctum. La otra posibilidad consiste en el **trabajo con tokens de API**, especialmente pensado en servicios web que no manejen estado (stateless).

Para explicar todo el flujo de **Autenticación con tokens para API usando Sanctum en Laravel** seguiremos los siguientes apartados.

- Consideraciones Stateful o Stateless
- Qué son los token de API
- Instalamos el proyecto Laravel
- Trait para trabajar con tokens en el modelo User
- Flujo para el registro de usuarios
- Flujo para login de usuarios
- Middleware 'auth:sanctum'
- Flujo para logout de usuarios en el API

Consideraciones Stateful o Stateless

En otras ocasiones ya hemos abordado la diferencia entre estos dos **métodos de trabajo para la construcción de APIs**. Puedes consultar el primer artículo dedicado a los Packages del ecosistema Laravel para construir un API.

De todos modos cabe decir que el **modelo stateful es únicamente adecuado cuando vamos a trabajar con un proyecto SPA** que depende directamente del sitio web donde va a ser consumido, ya sea en el mismo dominio o en un subdominio dependiente.

Si ya queremos **trabajar con el API desde cualquier dominio** posible, o incluso desde otras plataformas que no sean específicamente web, como **aplicaciones móviles o aplicaciones de escritorio**, lo adecuado es acudir al modelo Stateless. También resultará el modelo adecuado cuando necesitamos **altas o muy altas características de escalabilidad**, ya que un proyecto backend que trabaja con sesiones resulta más difícil de escalar, llegando a ser casi imposible de gestionar en un cluster de servidores.

Qué son los token de API

En el modelo stateless trabajamos con tokens de API para la autorización de los usuarios. Funcionan básicamente de la siguiente manera:

- Cuando un usuario se autentica envía sus datos de login normales
- Cuando el servidor Backend comprueba que son correctos, **devuelve un token** al usuario, que consiste en una cadena larga de caracteres.
- En los siguientes accesos al Backend el cliente enviará el token, verificando así que es el usuario autenticado

> Se puede ver más información a nivel general sobre este mecanismo de trabajo de token en el artículo siguiente: Autenticación por token.

Por suerte para nosotros **Laravel Sanctum gestiona perfectamente el modelo Stateless** y nos permite **trabajar con tokens de una manera muy sencilla**, como vamos a ver a continuación.

Instalamos el proyecto Laravel

Vamos a comenzar desde cero, creando un nuevo proyecto Laravel en el que implementaremos el trabajo con tokens de API.

La manera más sencilla actualmente para crear un nuevo proyecto Laravel es vía Sail, un mecanismo muy práctico para empezar a trabajar sin tener que preocuparse por el entorno y todos los servicios necesarios como las bases de datos.

```
curl -s "https://laravel.build/mi-proyecto-de-api?with=mariadb,redis,mailpit" | bash
```

En el comando anterior solamente hemos elegido algunos servicios de los disponibles al usar Sail, porque hay cosas que no las vamos a necesitar. Simplemente sustituyes `mi-proyecto-de-api` por el nombre de tu proyecto. Puedes seleccionar los servicios que prefieras. Por ejemplo en el comando anterior estamos definiendo que se use MariaDB en lugar de MySQL, pero esto es solo una preferencia personal y puedes cambiarlo si lo deseas.

Luego nos metemos en la carpeta que se ha generado con el proyecto y hacemos:

```
./vendor/bin/sail up
```

Pero es preferible crear el shortcut para hacer simplemente:

```
sail up
```

Todo el trabajo con contenedores Docker em Laravel, usando Sail, lo encuentras detallado en nuestra Guía de Laravel Sail.

Como Sanctum ya viene instalado en principio solamente hace falta correr las migraciones.

```
sail artisan migrate
```

Trait para trabajar con tokens en el modelo User

Para usar tokens para autenticación de usuarios deberíamos usar un trait en el modelo de usuarios (`User`) que está creado ya.

El trait es `Laravel\Sanctum\HasApiTokens`. Este trait ya se usa actualmente en el modelo, por lo que no necesitamos realizar cambios. Solamente lo queremos mencionar para que seas consciente de **las configuraciones existentes en el proyecto, de casa, que te permitirán usar los tokens para autenticación**.

Flujo para el registro de usuarios

Ahora vamos a ver todas las cosas que necesitamos incorporar a nuestra aplicación para poder trabajar con los Token. Todo este trabajo lo tenemos que configurar manualmente, ya que los starter kits que nos ofrecen en Laravel tienen código que está preparado para trabajar exclusivamente con el modelo stateful.

Ruta para el registro de usuarios

Creamos la ruta para permitir el **registro de usuarios de nuestro API** en el archivo de rutas de API: `routes/api.php`

```
Route::post('/auth/register', [RegisterController::class, 'registerUser']);
```

Controlador para registro de usuarios

Creamos el controlador `RegisterController` con Artisan.

```
php artisan make:controller Auth/RegisterController
```

> He creado el controlador de registro dentro de una carpeta llamada `Auth`. Esto no es necesario pero puede ayudarnos a mantener ordenados y agrupados todos los controladores del proceso de autenticación.

Pero recuerda que si estás en Sail tienes que ejecutar el comando así:

```
sail artisan make:controller Auth/RegisterController
```

> Esto de la personalización de los comandos para Sail lo damos ya por sabido, por lo que no lo volveré a mencionar en el artículo.

El código completo de este controlador lo podemos ver en el siguiente listado:

```
<?php

namespace App\Http\Controllers\Auth;

use App\Models\User;
use Illuminate\Http\Request;
use OpenApi\Annotations as OA;
use App\Http\Controllers\Controller;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Facades\Validator;

class RegisterController extends Controller
{

    private $registerValidationRules = [
        'name' => 'required',
        'email' => 'required|email|unique:users,email',
        'password' => 'required'
    ];

    public function registerUser(Request $request) {
        $validateUser = Validator::make($request->all(), $this->registerValidationRules);
```

```
        if($validateUser->fails()){
            return response()->json([
                'message' => 'Ha ocurrido un error de validación',
                'errors' => $validateUser->errors()
            ], 400);
        }

        $user = User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => Hash::make($request->password)
        ]);

        return response()->json([
            'message' => 'El usuario se ha creado',
            'token' => $user->createToken("API ACCESS TOKEN")->plainTextToken
        ], 200);
    }
}
```

Flujo para login de usuarios

Ahora vamos a ver cómo se implementaría toda la mecánica de **login de usuarios en nuestro API**.

Ruta para el login:

```
Route::post('/auth/login', [LoginController::class, 'loginUser']);
```

Controlador para login de usuarios en el API

Creamos el controlador `LoginController` con artisan:

```
php artisan make:controller Auth/LoginController
```

El código de este controlador lo encuentras a continuación:

```
<?php

namespace App\Http\Controllers\Auth;

use App\Models\User;
use Illuminate\Http\Request;
use App\Http\Controllers\Controller;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Validator;

class LoginController extends Controller
{
    private $loginValidationRules = [
        'email' => 'required|email',
        'password' => 'required'
    ];

    public function loginUser(Request $request) {
        $validateUser = Validator::make($request->all(), $this->loginValidationRules);

        if($validateUser->fails()){
            return response()->json([
                'message' => 'Error de validación',
```

```
                'errors' => $validateUser->errors()
            ], 401);
        }

        if(!Auth::attempt($request->only(['email', 'password']))){
            return response()->json([
                'message' => 'El email y el password no corresponden con alguno de los usuarios',
            ], 401);
        }

        $user = User::where('email', $request->email)->first();

        return response()->json([
            'message' => 'Login correcto',
            'token' => $user->createToken("API ACCESS TOKEN")->plainTextToken
        ], 200);
    }
}
```

Middleware 'auth:sanctum'

Otra cosa que nos debe de quedar muy clara es cómo autorizar el acceso a un endpoint solamente si accede un usuario autenticado dentro de la aplicación, **verificando el correspondiente token**. Esto se puede realizar de diversos modos, siendo el más directo el uso del middleware '`auth:sanctum`', que nos permite asegurar que ciertas rutas están solamente estén disponibles para usuarios debidamente autenticados.

Un ejemplo de ruta que usa este middleware lo vamos a usar para el logout.

Flujo para logout de usuarios en el API

Ahora veremos cómo realizar la operativa para cerrar la sesión de los usuarios, que tendrá básicamente que eliminar los Token asociados al usuario que hace el logout.

Ruta para logout

```
Route::middleware('auth:sanctum')->get('/auth/logout', [LogoutController::class, 'logoutUser']);
```

Controlador para cerrar la sesión y eliminar el token

Creamos el controlador `LogoutController`

```
php artisan make:controller Auth/LogoutController
```

El código completo del controlador `LogoutController` se puede ver a continuación.

```
<?php

namespace App\Http\Controllers\Auth;

use Illuminate\Http\Request;
use App\Http\Controllers\Controller;
use Illuminate\Support\Facades\Auth;

class LogoutController extends Controller
```

```
{
    public function logoutUser() {
        $user = Auth::user();
        $user->tokens()->delete();

        return response()->json([
            'message' => 'Has salido de la cuenta',
        ], 200);
    }
}
```

Otras operativas para el trabajo con token disponibles en Sanctum

En este artículo hemos visto lo más básico sobre los token para APIs. En realidad Laravel permite trabajar también con otro tipo de **operativas esenciales para API con necesidades un poco más avanzadas**.

Por ejemplo podemos crear tokens que tengan un tiempo de caducidad más o menos corto, crear tokens que ofrezcan "*scopes*" determinados, de modo que podamos autorizar o no algunas funcionalidades dependiendo del tipo de token expedido.

En caso de que las necesites te recomendamos leer la documentación de Laravel para encontrar más información sobre las cosas adicionales que puedes llegar a realizar con los token de API gracias a Sanctum.

Este artículo es obra de *Miguel Angel Alvarez*

Fue publicado / actualizado en *07/03/2024*

Disponible online en *https://desarrolloweb.com/articulos/autenticacion-token-api-laravel*